The sample data has information for two cameras, an `X100` and an `X1`. Each camera has a `description` and a `stock` count. Sure, the data could be used in a program without *any* classes, but classes buy you an abstraction by separating your data from your program.

5.1.2. Class abstractions

To learn why classes are useful, start without *any* abstraction and see what happens. Take the camera data and display each camera as a string suitable for display on the web page:

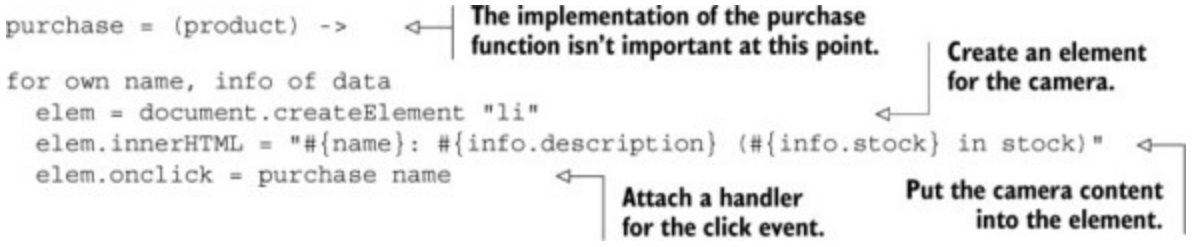
```
"X100: a camera (5 in stock)"
```

Then create an array of these strings using a comprehension (see section 4.3 if you need a refresher on comprehensions):

```
for own name, info of data
  "#{name}: #{info.description} (#{info.stock} in stock)"

# ["X100: A really cool camera (5 in stock)",
#  "X1: An awesome camera (6 in stock)"]
```

Now imagine that Agtron wants users to purchase cameras by clicking them. Modify the comprehension to create an element in the document, add the camera data, and then add a click handler. Doing this in a web browser looks like the following (note that the following example does not work in your REPL):



This is getting more complicated: there are more things for your brain to keep track of and more things for you to worry about. Regardless, you can struggle through, so far.